

CSCE 411

Divide and Conquer Practice

1. Give an algorithm which takes as input an array $A[1, \dots, n]$ of n *distinct* integers (some of which may be negative) sorted in increasing order and determines whether or not there exists an integer $1 \leq i \leq n$ such that $A[i] = i$. Your algorithm should run in $\Theta(\log n)$ time.

Solution: We give the following algorithm, which is a slight variant on binary search:

```
FINDINDEX( $A[1, \dots, n]$ ,  $low \leftarrow 1$ ,  $high \leftarrow n$ )
  If  $low > high$ :
    Return False
   $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
  If  $A[mid] = mid$ :
    Return True
  If  $A[mid] < mid$ :
    Return FINDINDEX( $A$ ,  $mid + 1$ ,  $high$ )
  Return FINDINDEX( $A$ ,  $low$ ,  $mid - 1$ )
```

Correctness: Our algorithm maintains the invariant that if there is an i for which $A[i] = i$, then $low \leq i \leq high$.

Clearly this is true at the start of the initial call to the algorithm, when low is 1 and $high$ is n . Assuming that it is true at the start of a call to the algorithm, we show that it will also be true at the start of any recursive call the algorithm makes. We note that because the elements of the array are distinct integers, $A[mid - j] \leq A[mid] - j$ for any $0 \leq j \leq mid - 1$, and $A[mid + j] \geq A[mid] + j$ for any $0 \leq j \leq n - mid$. Therefore, if $A[mid] < mid$, $A[i] < i$ for every $1 \leq i \leq mid$, so our invariant holds for the recursive call to $\text{FINDINDEX}(A, mid + 1, high)$. Similarly, if $A[mid] > mid$, then $A[i] > i$ for every $mid \leq i \leq n$, so our invariant holds for the recursive call to $\text{FINDINDEX}(A, low, mid - 1)$.

Finally, we note that by our invariant, if $low > high$, then the array does not contain an i for which $A[i] = i$, so when the algorithm returns false it is correct. If $A[mid] = mid$, then there is an i for which $A[i] = i$ (namely $i = mid$) so when the algorithm returns true it is correct. The algorithm will always return one of these values, because $high - low$ decreases by at least 1 with each recursive call, so the algorithm will reach one of these two bases cases after a finite number of calls (and each call does finite work outside of the recursion).

Runtime: Our algorithm has the same recurrence as binary search: $T(n) = T(n/2) + \Theta(1)$, so its runtime is also $\Theta(\log n)$.

2. Suppose that you are given a sorted array A of n distinct integers and an integer x . You know that x is in A and you want to find its index. That is, you know that there exists some $t \in \mathbb{Z}^+$ such that $A[t] = x$, and you want to find t . Give an $O(\log t)$ time algorithm to find the index t such that $A[t] = x$. (Note that we can determine this in $O(\log n)$ time using binary search, but this may not be fast enough, because we are not guaranteed that $\log n \in O(\log t)$).

Solution: The idea here is to use exponential search to allow us to find x more quickly if it is near the beginning of A , at the cost of spending about twice as long as binary search when x is not early in A .

We give the following algorithm:

```

SEARCH( $A[1, \dots, n]$ )
   $i \leftarrow 1$ 
  While  $A[i] < x$ 
     $i \leftarrow \min(2i, n)$ 
  Binary search  $A[(i/2) : i]$  for  $x$ 

```

Correctness: Since A is sorted, we know that if $A[i] < x$ for some i , $A[j] < x$ for every $1 \leq j < i$. Similarly, if $A[i] > x$, we know that $A[j] > x$ for every $i < j \leq n$. Thus, when the while loop terminates, we know that x is in the range $A[(i/2) : i]$, so we will successfully find it with binary search.

Runtime: The while loop repeatedly doubles i until i is greater than t . Thus, it runs for $O(\log t)$ iterations (since $\log_2 t$ is, by definition, the number of times we need to double 1 in order to get to t). When the while loop stops, i is between t and $2t$, so the range $A[(i/2) : i]$ has size at most t and we can binary search it in $O(\log t)$ time. Thus, the overall running time of the algorithm is $O(\log t)$.

3. Suppose that you are given as input an n -digit number in base-10 (represented as a string) and want to compute the same number in base-3 (in base-3 the only valid digits are 0, 1, and 2, and a string of digits $x_1 x_2 \dots x_n$ represents the number $\sum_{i=0}^{n-1} 3^i x_{n-i}$).
 - (a) Give a $\Theta(n^2)$ time algorithm for this problem. You do not have to prove the correctness of your algorithm, but you should carefully analyze the runtime.
 - (b) Now give an $o(n^2)$ time algorithm (Hint: Divide and conquer)
 - (c) Now give an $O(n \log^2 n)$ time algorithm for the problem (Hint: Use repeated squaring to avoid a recursive call)

Solution: (a) We give the following algorithm:

```

TOBASE3( $x = x_1x_2 \dots x_n$ )
  result  $\leftarrow \varepsilon$ 
  While  $x > 0$ :
    Prepend  $(x \bmod 3)$  to the beginning of result
     $x \leftarrow \lfloor x/3 \rfloor$ 
  Return result

```

We note that this algorithm works by calculating the digits of x 's base 3 representation one by one from right to left. The last base 3 digit of x is $x \bmod 3$, and the rest of the digits form the number $\lfloor x/3 \rfloor$.

Runtime: Since we divide x by 3 in each iteration of the while loop, the loop runs $\Theta(\log x)$ times (since $\log_3 x$ is the number of times we need to divide x by 3 to get down to 1). We note that $\Theta(\log x) = \Theta(n)$, since n , the number of base-10 digits of x , is $1 + \lfloor \log_{10} x \rfloor$.

Inside of the loop we need to calculate $x \bmod 3$ and $\lfloor x/3 \rfloor$. Since 3 is a constant, we can calculate both of these using the grade school long division algorithm (iterate through the digits from left to right, keeping track of a remainder while building the quotient)[†] in $\Theta(n)$ time. Prepending $(x \bmod 3)$ to result takes $O(n)$ time. Thus, our loop runs $\Theta(n)$ times, doing $\Theta(n)$ work per iteration, so the total runtime is $\Theta(n \cdot n) = \Theta(n^2)$.

[†] : Although citing “grade school long division” is sufficient for a proof, we reproduce the algorithm in full here for the curious student. **DIVIDEBY3** returns a pair (q, r) where $q = \lfloor x/3 \rfloor$ is the quotient and $r = x \bmod 3$ is the remainder.

```

DIVIDEBY3( $x = x_1x_2 \dots x_n$ )
   $q \leftarrow$  a string of  $n$  zeros
   $r \leftarrow 0$ 
  For  $i$  from 1 to  $n$ :
     $temp \leftarrow 10 \cdot r + x_i$ 
     $q_i \leftarrow \lfloor temp/3 \rfloor$ 
     $r \leftarrow temp \bmod 3$ 
  Return  $(q, r)$ 

```

We note that r is always at most 2, so the arithmetic operations in **DIVIDEBY3** are on values that fit in RAM model integer variables, and thus these arithmetic operations take constant time. Therefore, **DIVIDEBY3** runs in time $O(n)$ (the number of loop iterations).

Solution: (b) Inspired by Karatsuba's algorithm for fast multiplication, we will use a divide and conquer approach. To make the algorithm and analysis simpler, we will assume that n is a power of 2. If this is not the case, we can simply add leading 0s to the input until the number of digits is a power of 2—this will increase the number of digits by at most a factor of 2.

```

TOBASE3FAST( $x = x_1x_2 \dots x_n$ )
  If  $n = 1$ :
    Return the base 3 equivalent of  $x_1$ 
   $a \leftarrow x_1x_2 \dots x_{n/2}$ 
   $b \leftarrow x_{n/2+1} \dots x_n$ 
   $c \leftarrow$  a string consisting of a 1 followed by  $n/2$  0s.
  Return TOBASE3FAST( $a$ )  $\cdot$  TOBASE3FAST( $c$ ) + TOBASE3FAST( $b$ )

```

We note that all of the arithmetic on the final line is done in base 3. Our algorithm is using the fact that $x = a10^{n/2} + b$. We convert a , $10^{n/2}$, and b to base 3 recursively and then recombine them with base 3 multiplication and addition.

To analyze the runtime of our algorithm we note that it makes 3 recursive calls, each on subproblems of size $n/2$. Outside of the recursive calls, the algorithm performs multiplications and additions of base-3 numbers with $O(n)$ digits (converting x to base 3 increases the number of digits by only a constant factor, specifically $(\log_3 x)/(\log_{10} x)$). As shown in the notes, we can add two n -digit numbers in $\Theta(n)$ time (this was shown for base 10, but the same algorithm works for any base), and we can multiply two n -digit numbers in $\Theta(n \log n)$ time. Therefore the runtime of our algorithm outside of the recursive calls is $\Theta(n \log n)$, and our algorithm's runtime satisfies the following recurrence:

$$T(n) = 3T(n/2) + \Theta(n \log n).$$

By the master theorem, this solves to $T(n) = \Theta(n^{\log_2 3})$ (the same as the recurrence for Karatsuba's algorithm).

Solution: (c) In order to speed up the algorithm from part (b), we will eliminate the recursive call to compute $10^{n/2}$. Since we have already assumed (via padding with 0s) that n is a power of 2, $n/2$ will also be a power of 2, as will $n/4$, and so on, so we only need to calculate the base 3 representations of 10^{2^i} for i from 0 up through $\log_2 n$. We can do this with repeated squaring (similar to the technique we used to speed up exponentiation in lecture).

We give the following algorithm which takes an n which is a power of 2 and computes an array A where $A[i]$ contains the base 3 representation of 10^i for each i which is a power of 2 less than or equal to n (we use an array of size n to make our algorithms slightly shorter and cleaner, but if we cared about space complexity we could accomplish the same thing with an array of size $\log(n)$ where $A[i]$ stored the base 3 representation of 10^{2^i} instead of the base-3 representation of 10^i)

```

COMPUTEPOWERSOFTEN( $n$ )
 $i \leftarrow 1$ 
 $A \leftarrow$  an array of  $n$  empty strings
 $A[1] \leftarrow$  an array containing "101" //base 3 representation of ten
While  $i < n$ :
     $i \leftarrow 2i$ 
     $A[i] \leftarrow A[i/2] \cdot A[i/2]$ 
Return  $A$ 

```

The work of COMPUTEPOWERSOFTEN outside of the while loop is $\Theta(n)$ (the only non-constant time operation is creating an array of size n). We note that the while loop in COMPUTEPOWERSOFTEN runs $O(\log n)$ times and each iterations involves multiplying two numbers with $O(i)$ digits (The length of 10^i in base 10 is $i + 1$, so the length of 10^i in base 3 is $O(i)$ and in the loop $i \leq n$). Multiplying two $O(n)$ -digit numbers takes $O(n \log n)$ time. Thus the total runtime for the loop and therefore the algorithm is $O((\log n)n \log n) = O(n \log^2 n)$.

After running COMPUTEPOWERSOFTEN to create a global array A , we can use the following variation of the algorithm from part (b) to solve the problem:

```

TOBASE3FASTEST( $x = x_1x_2 \dots x_n$ )
If  $n = 1$ :
    Return the base 3 equivalent of  $x_1$ 
 $a \leftarrow x_1x_2 \dots x_{n/2}$ 
 $b \leftarrow x_{n/2+1} \dots x_n$ 
Return TOBASE3FASTEST( $a$ )  $\cdot$   $A[n/2]$  + TOBASE3FASTEST( $b$ )

```

This algorithm does $\Theta(n \log n)$ work outside of its recursive calls to create a and b and perform addition and multiplication on n -digit numbers. This time, however, we have only two recursive calls, so the algorithm satisfies the recurrence $T(n) = 2T(n/2) + \Theta(n \log n)$. By the master theorem, this solves to $T(n) = \Theta(n \log^2 n)$. Thus, combining the runtime for COMPUTEPOWERSOFTEN and TOBASE3FASTEST we get that the total runtime for our algorithm is $\Theta(n \log^2 n)$.

4. We say that an array $A[1, \dots, n]$ of integers is a **valley** if there exists some index i such that for every $1 \leq j < i$ we have $A[j] > A[j + 1]$, and for every $i \leq j < n$ we have $A[j] < A[j + 1]$. That is, a valley consists of a strictly decreasing sequence followed by a strictly increasing sequence. For example,

$$[7, 5, 4, -2, -7, -15, 1, 3, 10]$$

is a valley, but

$$[5, 6, 8, 4, 2]$$

is not. Give and analyze an algorithm which takes as input an array A which is a valley, as well as an integer x and returns either

- An index j such that $A[j] = x$, or
- -1 if x is not in A .

You may assume that all of the elements of A are distinct. Your algorithm must run in $O(\log n)$ time.

Solution: Our approach is based on binary search. We first show how to find the index of the bottom of the valley (the point where the array switches from increasing to decreasing). We give the following pseudocode:

```

FINDBOTTOM( $A[1, \dots, n]$ ,  $low \leftarrow 1$ ,  $high \leftarrow n$ )
  If  $low = high$ :
    Return  $low$ 
  If  $A[low] < A[low + 1]$ :
    Return  $low$ 
  If  $A[high - 1] > A[high]$ :
    Return  $high$ 
   $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
  If  $A[mid] > A[mid + 1]$ :
    Return FINDBOTTOM( $A$ ,  $mid + 1$ ,  $high$ )
  Return FINDBOTTOM( $A$ ,  $low$ ,  $mid$ )

```

Let b be the index of the bottom. To prove the correctness of our algorithm, we first show that for every recursive call we make, we always have $low \leq b \leq high$. For the initial call this is guaranteed because every valley must have a bottom somewhere. If $A[mid] > A[mid + 1]$, then $A[mid]$ must be before the bottom (since it is part of the decreasing sequence), so we have that $mid + 1 \leq b \leq high$. Otherwise, since the elements of A are distinct, we must have $A[mid] < A[mid + 1]$, meaning that $A[mid]$ is part of the increasing sequence, and thus $low \leq b \leq mid$.

It remains to show that when FINDBOTTOM returns low or $high$ it does so correctly. Since we know $low \leq b \leq high$, if $low = high$, we must have $b = low$. If $A[low] < A[low + 1]$, then $A[low]$ is part of the increasing sequence, so we have $b \leq low$, and therefore $b = low$. Similarly, if $A[high - 1] > A[high]$ then $A[high]$ is part of the decreasing sequence, so $high \leq b$, and thus $high = b$. Therefore, whenever the algorithm returns, it returns a correct answer.

We note that $high - low$ decreases by a factor of two with every recursive call made by FINDBOTTOM, and therefore the total number of recursive calls is $O(\log n)$. Since the work done outside of the recursion is constant for each call, the overall runtime of the algorithm is $O(\log n)$.

Finally, we describe how to use FINDBOTTOM to determine whether or not x is in A . Since $A[1, \dots, b]$ is sorted in decreasing order, we can use binary search to determine if x is in this subarray in $O(\log n)$ time. Since $A[b, \dots, n]$ is sorted in increasing order we can use binary search to determine if x is in this subarray in $O(\log n)$ time. If either of the calls to binary search finds x we return True, otherwise we return False. Since finding the bottom takes $O(\log n)$ time and each binary search also takes $\Theta(\log n)$ time, the overall runtime of our algorithm is $\Theta(\log n)$.